

Chapter 9: Methods in Java

Personal Notes

By now you can write loops, conditions, and patterns. But what happens when your program grows to 200 lines and the same logic keeps repeating? You wrap it inside a method.

Methods are how Java programs stay clean, reusable, and easy to fix. Every real Java program — small or large — is just a collection of methods working together. Even main itself is a method.

1. What is a Method?

A method is a named block of code that performs a specific task. Once written, you can run that block any time you want by just calling its name. You don't have to copy-paste the same logic again and again.

Think of a microwave button. The button labelled "Popcorn" is a method. Someone wrote the popcorn logic once (heat, time, power), labelled it, and now any time you press it, the same logic runs. You don't care how it works inside — you just call it.

A simple Java method looks like this:

```
public static void sayHello() {  
    System.out.println("Hello!");  
}
```

And to actually run it (call it):

```
sayHello();    // prints: Hello!
```

2. Why Use Methods?

Methods exist for four very practical reasons:

- Reusability — write the logic once, call it many times
- Readability — main() stays clean and easy to follow
- Maintainability — fix a bug in one place, not ten
- Organization — break a big problem into small named pieces

Without methods, a 1000-line program is one giant blob. With methods, it becomes a list of named steps that read almost like English.

Without methods

```
// Calculate area of 3 circles  
double a1 = 3.14 * 5 * 5;  
System.out.println(a1);  
double a2 = 3.14 * 7 * 7;
```

```
System.out.println(a2);
double a3 = 3.14 * 10 * 10;
System.out.println(a3);
```

With a method

```
public static void printArea(int r) {
    double area = 3.14 * r * r;
    System.out.println(area);
}

// Call it as many times as you want
printArea(5);
printArea(7);
printArea(10);
```

Cleaner, shorter, and if you ever want to change the formula, you change it in one place.

3. Anatomy of a Method

Every Java method has the same basic structure. Once you recognize the parts, every method anywhere looks familiar.

modifier	return-type	name	(parameters)
v	v	v	v
public static	int	addNumbers	(int a, int b) {
	int sum = a + b;		
	return sum;		<-- return statement
}			
^			^
method header			method body

Each part has a job:

- Modifier — who can use the method (public, private, etc.) and how (static or not)
- Return type — what kind of value the method gives back (int, double, void, String, etc.)
- Name — the label you use to call the method (should describe what it does)
- Parameters — input the method needs to do its job (can be empty)
- Body — the actual code inside { }
- Return statement — sends the result back; not needed if return type is void

Naming tip: Method names start with a lowercase letter and use camelCase. They should be verbs because methods DO things. Good: calculateTotal, isEmpty, sendEmail. Bad: total, x, doStuff.

4. Calling a Method

Writing a method is only half the work. To actually run it, you must call it. Calling a method means using its name followed by parentheses, with any required values inside.

```
public class Main {  
    public static void main(String[] args) {  
        greet();          // call greet  
        int sum = add(5, 3); // call add and store result  
        System.out.println(sum);  
    }  
  
    public static void greet() {  
        System.out.println("Hello!");  
    }  
  
    public static int add(int a, int b) {  
        return a + b;  
    }  
}
```

Output:

```
Hello!  
8
```

What actually happens

When Java hits a method call, it pauses the current method, runs the called method, and then comes back to where it left off.

```
main() runs  
|  
|---> call greet()  
|       runs greet  
|       prints "Hello!"  
|       <--- returns  
|  
|---> call add(5, 3)  
|       runs add  
|       returns 8  
|       <--- returns 8  
|  
|       sum = 8  
|       print sum  
|  
end
```

5. Parameters vs Arguments

These two words mean almost the same thing but they're used in different places. Beginners mix them up a lot.

- Parameters are the variables listed in the method definition (the input slots)
- Arguments are the actual values you pass when calling the method

```
// 'a' and 'b' are parameters
public static int add(int a, int b) {
    return a + b;
}

// 5 and 3 are arguments
int result = add(5, 3);
```

Easy way to remember: Parameters are the empty boxes in the method header. Arguments are the actual values you drop into those boxes when calling.

6. Return Types

Every method must declare what kind of value it returns. If the method doesn't return anything, you use the special keyword `void`.

6.1 Methods that return a value

```
public static int square(int n) {
    return n * n;
}

int result = square(6);    // result = 36
```

The `return` statement does two things: it sends the value back, and it ends the method immediately. Any code after `return` inside the same block will NOT run.

6.2 Void methods

A void method does something but doesn't give a value back. Good examples are methods that just print or update something.

```
public static void printLine() {
    System.out.println("-----");
}

printLine();    // works
int x = printLine();    // ERROR – void methods can't be assigned
```

6.3 Return type must match

Whatever you return must match the declared return type. Java is strict about this.

```
public static int getNumber() {  
    return "hello";    // ERROR – String returned, but int expected  
}
```

Common bug: Forgetting to return a value in a non-void method gives a compile error: "missing return statement." Every possible path through the method must return something.

7. Method Overloading

Java lets you have multiple methods with the SAME name as long as their parameter lists are different. This is called method overloading.

It's useful when you want to do basically the same thing but with different inputs. The classic example is `println` — it works with `int`, `double`, `String`, `boolean`, and many more, all under the same name.

Example

```
public static int add(int a, int b) {  
    return a + b;  
}  
  
public static double add(double a, double b) {  
    return a + b;  
}  
  
public static int add(int a, int b, int c) {  
    return a + b + c;  
}  
  
// All three are valid and live together  
add(2, 3);           // calls first one  
add(2.5, 3.5);       // calls second one  
add(1, 2, 3);        // calls third one
```

Rules for overloading

Two methods can share a name ONLY if at least one of these is different:

- Number of parameters
- Type of parameters
- Order of parameter types

Important: Changing only the return type is NOT overloading. Java decides which overload to call based on the arguments, not the return type, so two methods with the same parameters but different return types will fail to compile.

8. Pass by Value (Important!)

When you call a method with arguments, Java copies those values into the method's parameters. The original variables outside the method are NOT shared — only their values are copied in.

This is called pass by value, and Java uses it for everything.

Example with a primitive

```
public static void changeIt(int x) {  
    x = 100;    // changes the local copy only  
}  
  
public static void main(String[] args) {  
    int a = 5;  
    changeIt(a);  
    System.out.println(a);    // still 5, not 100  
}
```

```
Before call:    a = 5    (in main)  
At call:        x = 5    (copy of a, inside changeIt)  
After x = 100:  x = 100  (still just the copy)  
Back in main:  a = 5    (untouched)
```

What about objects?

Objects are slightly trickier. Java still passes by value, but the value being copied is the REFERENCE to the object. So both the original variable and the parameter point to the same object.

```
public static void rename(Student s) {  
    s.name = "Updated";    // changes the actual object  
    s = null;               // changes only local copy of reference  
}
```

Inside the method, changing the object's fields IS visible outside. But reassigning the parameter itself is NOT, because the reference variable is just a copy.

Key idea: Java is always pass by value. For primitives, the value is the actual number. For objects, the value is the reference address. Either way, the method only gets a copy.

9. Variable Scope in Methods

Variables declared inside a method exist ONLY inside that method. They are called local variables. Once the method finishes, they vanish.

```
public static void demo() {  
    int x = 10;        // local to demo  
    System.out.println(x);  
}
```

```
public static void main(String[] args) {  
    demo();  
    System.out.println(x);    // ERROR – x doesn't exist here  
}
```

Two methods can have variables with the SAME name without conflict, because each method has its own world.

```
public static void methodA() {  
    int count = 5;  
}  
  
public static void methodB() {  
    int count = 99;    // totally separate from methodA's count  
}
```

10. Static vs Instance Methods (Quick Intro)

You've seen the keyword `static` in every method so far. There's a reason for that — and you'll meet the alternative when you reach OOP.

Static methods

Belong to the class itself. You call them using the class name (or directly inside the same class).

```
public class MathHelper {  
    public static int square(int n) {  
        return n * n;  
    }  
}  
  
int result = MathHelper.square(5);    // no object needed
```

Instance methods

Belong to an object. You must create the object first, then call the method on it.

```
public class Student {  
    String name;  
  
    public void greet() {        // notice: no 'static'  
        System.out.println("Hi, " + name);  
    }  
}  
  
Student s = new Student();  
s.name = "Aman";  
s.greet();    // call on the object
```

For now: Use static while you're learning. Once you reach classes and objects in detail, instance methods will make more sense. Just know both exist.

11. Quick Reference Table

Term	Meaning	Quick Example
void	Method returns nothing	<code>void printHello()</code>
int / double / etc.	Method returns that type	<code>int add(int a, int b)</code>
Parameters	Empty input slots in definition	<code>(int a, int b)</code>
Arguments	Actual values passed in call	<code>add(5, 3)</code>
return	Sends value back, ends method	<code>return a + b;</code>
Overloading	Same name, different params	<code>add(int,int) + add(double,double)</code>
Pass by value	Java copies the value in	Primitives stay unchanged outside
Local variable	Lives only inside its method	<code>int x</code> inside <code>doStuff()</code>

12. Practice Problems

Try these in your IDE. Each one builds a different method skill.

1. Write a method `isEven(int n)` that returns true if n is even, false otherwise.
2. Write a method `max(int a, int b)` that returns the larger of two numbers.
3. Write a method `factorial(int n)` that returns n! (use a loop, not recursion).
4. Write three overloaded sum methods: `sum(int, int)`, `sum(int, int, int)`, `sum(double, double)`.
5. Write a void method `printTable(int n)` that prints the multiplication table of n from 1 to 10.
6. Write a method `countDigits(int n)` that returns how many digits n has.
7. Write a method `reverse(int n)` that returns the digits of n reversed (123 → 321).
8. Write a method `isPrime(int n)` that returns true if n is prime.

13. Common Beginner Mistakes

Mistake 1: Forgetting parentheses when calling

```
greet;    // wrong – no parentheses
greet();  // right
```


Mistake 2: Defining a method inside main

In Java, methods are defined inside the class, NOT inside another method. You can CALL methods from main, but you cannot DEFINE them there.

```
public class Main {  
    public static void main(String[] args) {  
        // WRONG: can't define a method here  
    }  
  
    public static void greet() {    // correct place  
        System.out.println("Hi");  
    }  
}
```

Mistake 3: Expecting changes to primitives to stick

Java copies primitives. The original outside the method does not change. (See Section 8.)

Mistake 4: Missing return statement

```
public static int getMax(int a, int b) {  
    if (a > b) return a;  
    // forgot to handle the else case → compile error  
}
```

Mistake 5: Wrong return type

```
public static int give() {  
    return 3.5;    // ERROR – double returned where int expected  
}
```

14. Best Practices

- One method should do ONE thing. If it's doing too much, break it up.
- Keep methods short — ideally under 20 lines.
- Use clear, descriptive names. calculateTotal beats doIt.
- Use parameters instead of hardcoded values inside the method.
- Avoid more than 3-4 parameters; if you need more, you may need an object instead.
- Don't repeat yourself — if you copy-paste code more than twice, it should probably be a method.

15. What's Next?

Methods are the bridge to all the bigger Java topics. From here, the natural next steps are:

- Recursion — methods that call themselves

- Arrays and how to pass them to methods
- Classes and objects — where instance methods live
- Constructors — a special kind of method that creates objects

Key Takeaway: Methods are how you turn long, repetitive code into clean, reusable building blocks. Once you start thinking in methods, every problem becomes "what small named steps does this break into?" — and that mindset is the foundation of every real programming career.